

RDS TASK

Assignment Overview

This document covers ten sequential tasks that build a self-managed MariaDB database on EC2, populate and back it up, migrate it into an Amazon RDS MariaDB instance, then repeat the same pattern with the MySQL engine before configuring Multi-AZ, snapshot backup and restore, and a read replica on the RDS MySQL instance. Each task builds on the resources created in the tasks before it and is documented with its own steps, validation, explanation, and real-time use case.

The following resource reference is used throughout this document:

Task	Resource
1	MariaDB server on EC2 - self-managed source database engine
2	employees table + sample rows - dataset used for backup and migration testing
3	mysqldump logical backup file - portable export of schema and data
4	RDS instance (MariaDB engine) - managed target database for migration
5	Restored labdb on RDS - migrated copy of the EC2 dataset
6	MySQL server on EC2 - second self-managed source engine
7	RDS instance (MySQL engine) - managed target for the MySQL workflow
8	Multi-AZ standby - synchronous failover replica in a second Availability Zone
9	Manual snapshot + restored instance - point-in-time backup and recovery
10	Read replica - asynchronous read-scaling copy of the primary

Task 1: Create a MariaDB Database on EC2

Outcome

MariaDB server is installed and running on the EC2 instance, and a working database with one table exists and is ready to receive data.

Objective

To stand up a fully functional, self-managed relational database engine on EC2 as the source system for the backup and RDS migration exercises that follow.

Prerequisites

- A running EC2 instance (Amazon Linux 2023 or Amazon Linux 2).
- SSH or Session Manager access to the instance.
- Security group allowing inbound SSH (port 22), and port 3306 if remote client access is needed.

Step-by-Step Implementation

Step 1: Install and Start MariaDB Server

```
sudo yum update -y
sudo yum install mariadb105-server -y
sudo systemctl start mariadb
sudo systemctl enable mariadb
sudo systemctl status mariadb
```

Screenshot Evidence:

```
[root@ip-172-31-21-168 ec2-user]# yum install mariadb105-server -y
Last metadata expiration check: 0:03:06 ago on Mon Aug 3 12:51:03 2026.
Dependencies resolved.
=====
Package                               Arch    Version                               Repository    Size
=====
Installing:
mariadb105-server                     x86_64  3:10.5.29-1.amzn2023.0.1            amazonlinux   10 M
Installing dependencies:
mariadb-connector-c                   x86_64  3.3.10-1.amzn2023.0.2                amazonlinux   211 k
mariadb-connector-c-config            noarch  3.3.10-1.amzn2023.0.2                amazonlinux   9.3 k
mariadb105                            x86_64  3:10.5.29-1.amzn2023.0.1            amazonlinux   1.5 M
mariadb105-common                     x86_64  3:10.5.29-1.amzn2023.0.1            amazonlinux   28 k
mariadb105-errmsg                     x86_64  3:10.5.29-1.amzn2023.0.1            amazonlinux   212 k
mysql-selinux                         noarch  1.0.14-2.amzn2023.0.1                amazonlinux   36 k
perl-DBD-MariaDB                     x86_64  1.22-1.amzn2023.0.5                  amazonlinux   152 k
perl-DBI                             x86_64  1.650-1.amzn2023.0.1                 amazonlinux   808 k
perl-File-Copy                        noarch  2.34-477.amzn2023.0.9                amazonlinux   20 k
perl-Math-BigInt                      noarch  1:1.9998.39-2.amzn2023.0.2           amazonlinux   202 k
perl-Math-BigRat                      noarch  0.2624-500.amzn2023.0.2              amazonlinux   42 k
perl-Math-Complex                     noarch  1.59-477.amzn2023.0.9                amazonlinux   47 k
perl-Module-Load                      noarch  1:0.36-2.amzn2023.0.2                amazonlinux   18 k
perl-Sys-Hostname                     x86_64  1.23-477.amzn2023.0.9                amazonlinux   17 k
Installing weak dependencies:
mariadb105-backup                     x86_64  3:10.5.29-1.amzn2023.0.1            amazonlinux   6.0 M
mariadb105-cracklib-password-check    x86_64  3:10.5.29-1.amzn2023.0.1            amazonlinux   13 k
mariadb105-gssapi-server              x86_64  3:10.5.29-1.amzn2023.0.1            amazonlinux   15 k
mariadb105-server-utils               x86_64  3:10.5.29-1.amzn2023.0.1            amazonlinux   207 k
=====
Transaction Summary
=====
Install 19 Packages

Total download size: 20 M
Installed size: 117 M
Downloading Packages:
(1/19): mariadb-connector-c-config-3.3.10-1.amzn2023.0.2.x86_64.rpm 215 kB/s | 9.3 kB 00:00
(2/19): mariadb105-10.5.29-1.amzn2023.0.1.x86_64.rpm                28 MB/s | 1.5 MB 00:00
(3/19): mariadb-connector-c-3.3.10-1.amzn2023.0.2.x86_64.rpm         3.3 MB/s | 211 kB 00:00
(4/19): mariadb105-common-10.5.29-1.amzn2023.0.1.x86_64.rpm          976 kB/s | 28 kB 00:00
Verifying : perl-DBI-1.650-1.amzn2023.0.1.x86_64                    13/19
Verifying : perl-File-Copy-2.34-477.amzn2023.0.9.noarch              14/19
Verifying : perl-Math-BigInt-1:1.9998.39-2.amzn2023.0.2.noarch       15/19
Verifying : perl-Math-BigRat-0.2624-500.amzn2023.0.2.noarch          16/19
Verifying : perl-Math-Complex-1.59-477.amzn2023.0.9.noarch           17/19
Verifying : perl-Module-Load-1:0.36-2.amzn2023.0.2.noarch            18/19
Verifying : perl-Sys-Hostname-1.23-477.amzn2023.0.9.x86_64           19/19
Installed:
mariadb-connector-c-3.3.10-1.amzn2023.0.2.x86_64
mariadb-connector-c-config-3.3.10-1.amzn2023.0.2.noarch
mariadb105-3:10.5.29-1.amzn2023.0.1.x86_64
mariadb105-backup-3:10.5.29-1.amzn2023.0.1.x86_64
mariadb105-common-3:10.5.29-1.amzn2023.0.1.x86_64
mariadb105-cracklib-password-check-3:10.5.29-1.amzn2023.0.1.x86_64
mariadb105-errmsg-3:10.5.29-1.amzn2023.0.1.x86_64
mariadb105-gssapi-server-3:10.5.29-1.amzn2023.0.1.x86_64
mariadb105-server-3:10.5.29-1.amzn2023.0.1.x86_64
mariadb105-server-utils-3:10.5.29-1.amzn2023.0.1.x86_64
mysql-selinux-1.0.14-2.amzn2023.0.1.noarch
perl-DBD-MariaDB-1.22-1.amzn2023.0.5.x86_64
perl-DBI-1.650-1.amzn2023.0.1.x86_64
perl-File-Copy-2.34-477.amzn2023.0.9.noarch
perl-Math-BigInt-1:1.9998.39-2.amzn2023.0.2.noarch
perl-Math-BigRat-0.2624-500.amzn2023.0.2.noarch
perl-Math-Complex-1.59-477.amzn2023.0.9.noarch
perl-Module-Load-1:0.36-2.amzn2023.0.2.noarch
perl-Sys-Hostname-1.23-477.amzn2023.0.9.x86_64
Complete!
```

Step 2: Secure the Installation

```
sudo mysql_secure_installation
```

Screenshot Evidence:

```
[root@ip-172-31-21-168 ec2-user]# sudo mysql_secure_installation
```

```
NOTE: RUNNING ALL PARTS OF THIS SCRIPT IS RECOMMENDED FOR ALL MariaDB  
SERVERS IN PRODUCTION USE! PLEASE READ EACH STEP CAREFULLY!
```

```
In order to log into MariaDB to secure it, we'll need the current  
password for the root user. If you've just installed MariaDB, and  
haven't set the root password yet, you should just press enter here.
```

```
Enter current password for root (enter for none):  
OK, successfully used password, moving on...
```

```
Setting the root password or using the unix_socket ensures that nobody  
can log into the MariaDB root user without the proper authorisation.
```

```
You already have your root account protected, so you can safely answer 'n'.
```

```
Switch to unix_socket authentication [Y/n] n  
... skipping.
```

```
You already have your root account protected, so you can safely answer 'n'.
```

```
Change the root password? [Y/n] admin12345
```

```
You already have your root account protected, so you can safely answer 'n'.
```

```
Change the root password? [Y/n] y
```

```
New password:
```

```
Re-enter new password:
```

```
Sorry, passwords do not match.
```

```
New password:
```

```
Re-enter new password:
```

```
Password updated successfully!
```

```
Reloading privilege tables..
```

```
... Success!
```

```
By default, a MariaDB installation has an anonymous user, allowing anyone  
to log into MariaDB without having to have a user account created for  
them. This is intended only for testing, and to make the installation  
go a bit smoother. You should remove them before moving into a  
production environment.
```

```
Remove anonymous users? [Y/n] n
```

```
... skipping.
```

```
New password:
Re-enter new password:
Password updated successfully!
Reloading privilege tables..
... Success!

By default, a MariaDB installation has an anonymous user, allowing anyone
to log into MariaDB without having to have a user account created for
them. This is intended only for testing, and to make the installation
go a bit smoother. You should remove them before moving into a
production environment.

Remove anonymous users? [Y/n] n
... skipping.

Normally, root should only be allowed to connect from 'localhost'. This
ensures that someone cannot guess at the root password from the network.

Disallow root login remotely? [Y/n] n
... skipping.

By default, MariaDB comes with a database named 'test' that anyone can
access. This is also intended only for testing, and should be removed
before moving into a production environment.

Remove test database and access to it? [Y/n] n
... skipping.

Reloading the privilege tables will ensure that all changes made so far
will take effect immediately.

Reload privilege tables now? [Y/n] y
... Success!

Cleaning up...

All done! If you've completed all of the above steps, your MariaDB
installation should now be secure.

Thanks for using MariaDB!
[root@ip-172-31-21-168 ec2-user]# |
```

Step 3: Create the Database and Table

```
mysql -u root -p
CREATE DATABASE khadeer;
USE khadeer;
CREATE TABLE employees (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50),
  department VARCHAR(50),
  salary DECIMAL(10,2)
);
```

Screenshot Evidence:

```
[root@ip-172-31-21-168 ec2-user]# mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 8
Server version: 10.5.29-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
+-----+
3 rows in set (0.000 sec)
```

```
MariaDB [(none)]> create database khadeer
-> ;
-> ;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near ':' at line 2
MariaDB [(none)]> create database khadeer;
Query OK, 1 row affected (0.000 sec)

MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| khadeer |
| mysql |
| performance_schema |
+-----+
4 rows in set (0.000 sec)
```

```
MariaDB [(none)]> use khadeer;
Database changed
MariaDB [khadeer]> create table employees (id INT PRIMARY KEY AUTO_INCREMENT,
->   name VARCHAR(50),
->   department VARCHAR(50),
->   salary DECIMAL(10,2)
-> );
Query OK, 0 rows affected (0.008 sec)

MariaDB [khadeer]> select * from employees
-> ;
Empty set (0.001 sec)

MariaDB [khadeer]> desc employees
-> ;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id | int(11) | NO | PRI | NULL | auto_increment |
| name | varchar(50) | YES | | NULL | |
| department | varchar(50) | YES | | NULL | |
| salary | decimal(10,2) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.001 sec)

MariaDB [khadeer]>
```

Validation Steps

- SHOW DATABASES; lists labdb.
- USE labdb; followed by SHOW TABLES; lists employees.

- DESCRIBE employees; shows the four defined columns with correct data types.

Explanation

MariaDB is a community-developed, drop-in-compatible fork of MySQL. Installing it directly on EC2 gives full control over the operating system, configuration files, and storage, but the operator is responsible for patching, backup scheduling, and high availability. This manually-managed setup is the baseline against which the managed RDS instance in Task 4 is later compared.

Real-Time Use Case

Teams migrating a legacy on-premises or self-hosted database into AWS typically first replicate the existing self-managed setup on an EC2 instance so behavior, data types, and application compatibility can be validated before cutting over to a managed RDS target.

Common Errors and Troubleshooting

Error	Cause	Resolution
Access denied for user 'root'@'localhost'	mysql_secure_installation was skipped or a different auth plugin is active	Re-run sudo mysql_secure_installation and set a root password, or connect with sudo mysql
Unit mariadb.service could not be found	Package installed under a different service name	Check with systemctl list-units --type=service grep -i maria and use the matching unit name

Task 2: Insert Dummy Data

Outcome

The employees table on the EC2 MariaDB instance is populated with sample rows representing typical application data.

Objective

To create a realistic, non-trivial dataset so that backup, restore, and migration procedures in later tasks operate on real rows rather than an empty schema.

Prerequisites

- Task 1 completed (labdb database and employees table exist).
- MySQL client access to the local MariaDB instance.

Step-by-Step Implementation

Step 1: Insert Sample Rows

```
INSERT INTO employees (name, department, salary) VALUES
('Ravi Kumar', 'Engineering', 65000.00),
('Ananya Rao', 'Finance', 58000.00),
('Suresh Reddy', 'Engineering', 72000.00),
```

```
('Divya Sharma', 'HR', 49000.00),  
( 'Kiran Patil', 'Sales', 54000.00);
```

Screenshot Evidence:

```
MariaDB [khadeer]> INSERT INTO employees (name, department, salary) VALUES  
-> ('Ravi Kumar', 'Engineering', 65000.00),  
-> ('Ananya Rao', 'Finance', 58000.00),  
-> ('Suresh Reddy', 'Engineering', 72000.00),  
-> ('Divya Sharma', 'HR', 49000.00),  
-> ('Kiran Patil', 'Sales', 54000.00);
```

Query OK, 5 rows affected (0.003 sec)

Records: 5 Duplicates: 0 Warnings: 0

```
MariaDB [khadeer]> select * from employees;
```

id	name	department	salary
1	Ravi Kumar	Engineering	65000.00
2	Ananya Rao	Finance	58000.00
3	Suresh Reddy	Engineering	72000.00
4	Divya Sharma	HR	49000.00
5	Kiran Patil	Sales	54000.00

5 rows in set (0.000 sec)

Validation Steps

- SELECT COUNT(*) FROM employees; returns a non-zero row count matching what was inserted.
- SELECT * FROM employees LIMIT 5; returns readable sample rows with correct column values.

Explanation

Populating the table before taking a backup ensures the exercise proves that both schema (DDL) and data (DML) are captured and restored correctly, not just an empty table structure.

Real-Time Use Case

Development and QA teams routinely seed tables with representative sample data before running migration or disaster-recovery drills, since an empty-table test would not catch data-type mismatches or encoding issues that only appear with real values.

Task 3: Take a Backup of the Dummy Data on EC2

Outcome

A logical backup file (labdb_backup.sql) containing the full schema and data of labdb is created and stored on the EC2 instance.

Objective

To produce a portable, text-based export of the database that can later be replayed against a different database engine or host, specifically the RDS instance created in Task 4.

Prerequisites

- Task 1 and Task 2 completed (khadeer populated with data).
- mysqldump utility available (installed with the MariaDB client package).

Step-by-Step Implementation

Step 1: Run mysqldump

```
mysqldump -u root -p khadeer > khadeer_backup.sql
```

Screenshot Evidence:

```
MariaDB [khadeer]> exit
Bye
[root@ip-172-31-21-168 ec2-user]# mysqldump -u root -p khadeer > khadeer_backup.sql
Enter password:
```

Step 2: Verify the Backup File

Confirm the file has a non-zero size and its contents include CREATE TABLE and INSERT INTO statements for the employees table.

```
ls -lh khadeer_backup.sql
head -n 20 labdb_backup.sql
```

Screenshot Evidence:

```
MariaDB [khadeer]> exit
Bye
[root@ip-172-31-21-168 ec2-user]# mysqldump -u root -p khadeer > khadeer_backup.sql
Enter password:
[root@ip-172-31-21-168 ec2-user]# ls -lh khadeer_backup.sql
-rw-r--r--. 1 root root 2.3K Aug  3 13:14 khadeer_backup.sql
[root@ip-172-31-21-168 ec2-user]# |
```

Validation Steps

- khadeer_backup.sql exists and its file size is greater than zero bytes.
- The file contains a CREATE TABLE employees statement and one or more INSERT INTO employees statements.

Explanation

mysqldump exports the database as a sequence of plain SQL statements (a logical backup). Because it is just SQL text, it is portable across MySQL and MariaDB versions and engines, unlike a storage-level snapshot, which is tied to the exact engine and version that produced it.

Real-Time Use Case

Logical backups of this kind are the standard mechanism for moving data between different database platforms or major versions where binary snapshot restore is not compatible, which is exactly the mechanism used in Task 5 to migrate the dataset into RDS.

Common Errors and Troubleshooting

Error	Cause	Resolution
mysqldump: Got error: 1044 Access denied	The database user lacks SELECT/LOCK TABLES privileges on labdb	Run the dump as root or grant the required privileges: GRANT SELECT, LOCK TABLES ON labdb.* TO 'user'@'localhost';
Resulting file is 0 bytes	Output redirection failed or command was run against the wrong database name	Re-run the command and confirm the database name and shell redirection syntax

Task 4: Launch a MariaDB RDS Instance

Outcome

A managed Amazon RDS instance running the MariaDB engine is created and reaches the Available state, ready to receive the migrated dataset.

Objective

To provision a managed relational database service equivalent to the self-managed EC2 database in Task 1, shifting patching, storage management, and backup scheduling to AWS.

Prerequisites

- IAM permissions to create RDS instances, subnet groups, and security groups.
- A VPC with at least two subnets in different Availability Zones for the DB subnet group.

Step-by-Step Implementation

Step 1: Open the RDS Create Database Wizard

RDS Console -> Databases -> Create database -> Standard create, then configure:

Parameter	Value
Engine type	MariaDB
Engine version	Match or exceed the EC2 source version
Templates	Dev/Test (or Free tier if eligible)
DB instance identifier	mariadb-rds
Master username	admin
DB instance class	db.t3.micro
Storage	20 GiB, gp2
VPC	Same VPC as the EC2 instance
Public access	No (or Yes, for lab convenience)
VPC security group	Allow inbound 3306 from the EC2 instance's security group

Screenshot Evidence:

[Aurora and RDS](#) > [Databases](#) > Create database

Settings

Engine version [Info](#)
View the engine versions that support the following database features.

▼ **Hide filters**

☐ Show only versions that support the Amazon RDS Optimized Writes [Info](#)
Amazon RDS Optimized Writes improves write throughput by up to 2x at no additional cost.

Engine version

MariaDB 11.8.8 ▼

DB instance identifier [Info](#)
Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in the current AWS Region.

mariadb-rds

The DB instance identifier is case-insensitive, but is stored as all lowercase (as in "mydbinstance"). Constraints: 1 to 63 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

Credentials Settings

Master username [Info](#)
Type a login ID for the master user of your DB instance.

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

Instance configuration

The DB instance configuration options below are limited to those supported by the engine that you selected above.

DB instance class [Info](#)

▼ **Hide filters**

☐ Show instance classes that support Amazon RDS Optimized Writes [Info](#)
Amazon RDS Optimized Writes improves write throughput by up to 2x at no additional cost.

☐ Include previous generation classes

☐ Standard classes (includes m classes)

☐ Memory optimized classes (includes r and x classes)

☒ Burstable classes (includes t classes)

Instance type

db.t4g.micro
2 vCPUs 1 GiB RAM EBS Bandwidth: Up to 2,085 Mbps Network: Up to 5 Gbps ▼

Connectivity

[Info](#)

Compute resource
Choose whether to set up a connection to a compute resource for this database. Setting up a connection will automatically change connectivity settings so that the compute resource can connect to this database.

☒ Don't connect to an EC2 compute resource
Don't set up a connection to a compute resource for this database. You can manually set up a connection to a compute resource later.

☐ Connect to an EC2 compute resource
Set up a connection to an EC2 compute resource for this database.

Network type [Info](#)
To use dual-stack mode, make sure that you associate an IPv6 CIDR block with a subnet in the VPC you specify.

☒ IPv4
Your resources can communicate only over the IPv4 addressing protocol.

☐ Dual-stack mode
Your resources can communicate over IPv4, IPv6, or both.

Virtual private cloud (VPC) [Info](#)
Choose the VPC. The VPC defines the virtual networking environment for this DB instance.

Default VPC (vpc-0a701668a4ff9329c)
6 Subnets, 6 Availability Zones ▼

After a database is created, you can't change its VPC. Only VPCs with a corresponding DB subnet group are listed.

ⓘ After a database is created, you can't change its VPC.

Step 2: Wait for the Instance to Become Available

Instance creation typically takes several minutes. Monitor progress in the RDS console until status changes to Available, then note the endpoint hostname from the Connectivity & security tab.

Screenshot Evidence:

Connectivity & security		
Endpoint & port	Networking	Security
Endpoint  mariadb-rds.cy1guuygysx7.us-east-1.rds.amazonaws.com	Availability Zone us-east-1a VPC vpc-0a701668a4ff9329c Subnet group default-vpc-0a701668a4ff9329c Subnets subnet-0c7b1347a08f35991 subnet-07ec013c768732ac7 subnet-0e09481a03600fe5c subnet-06485468739afd5a8 subnet-06c0f69df529bbfe4 subnet-0eb03781d4de1bf5b	VPC security groups mysg (sg-08c0ae7970e16fa6d)  Active Publicly accessible No Certificate authority Info rds-ca-rsa2048-g1 Certificate authority date May 26, 2061, 05:04 (UTC+05:30) DB instance certificate expiration date August 03, 2027, 18:53 (UTC+05:30)
	Network type	

Validation Steps

- The instance status shows Available in the RDS console.
- The endpoint and port (3306) are visible under the Connectivity & security tab.
- telnet <rds-endpoint> 3306 (or a mysql client connection attempt) from the EC2 instance succeeds.

Explanation

RDS automates provisioning, OS and engine patching, and underlying storage management for a managed relational engine, removing the operational burden of the manual installation performed in Task 1 while remaining wire-compatible with standard MariaDB clients and tools.

Real-Time Use Case

Organizations moving off self-managed database servers use RDS to eliminate patching and backup-script maintenance overhead while keeping the same SQL dialect and client tools their applications already use.

Common Errors and Troubleshooting

Error	Cause	Resolution
Cannot connect: timeout	Security group does not allow inbound 3306 from the EC2 instance	Edit the RDS security group inbound rules to allow port 3306 from the EC2 instance's security group or private IP
Instance stuck in Creating for a long time	Normal provisioning time for the selected instance class and storage type	Wait for provisioning to complete; large storage or Multi-AZ selections take longer

Task 5: Migrate the Database from EC2 to RDS

Outcome

The labdb schema and all rows from the EC2 source are present and queryable on the RDS MariaDB instance, matching the source row counts.

Objective

To move the dataset created and backed up in Tasks 1-3 into the managed RDS instance created in Task 4, completing the migration from self-managed to managed database.

Prerequisites

- Task 3 completed (labdb_backup.sql exists on the EC2 instance).
- Task 4 completed (RDS instance Available and reachable on port 3306 from EC2).

Step-by-Step Implementation

Step 1: Login to the Rds by using endpoint And create a table khadeer

```
mysql -h <rds-endpoint> -u admin -p
show databases;
create database;
```

Screenshot Evidence:

```
[ec2-user@ip-172-31-21-168 ~]$ mysql -h mariadb-rds.cylguuygysx7.us-east-1.rds.amazonaws.com -P 3306 -u admin -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 34
Server version: 11.8.8-MariaDB-log managed by https://aws.amazon.com/rds/

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| innodb |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.005 sec)

MariaDB [(none)]> create database khadeer;
Query OK, 1 row affected (0.005 sec)

MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| innodb |
| khadeer |
| mysql |
| performance_schema |
| sys |
+-----+
6 rows in set (0.004 sec)
```

Step 2: Restore the Dump File into RDS

This replays the CREATE DATABASE, CREATE TABLE, and INSERT statements from the backup file directly against the RDS endpoint.

```
mysql -h <rds-endpoint> -P 3306 -u <user_name> -p database_name< khadeer_backup.sql
```

Step 3: Verify the Migrated Data

```
mysql -h <rds-endpoint> -u admin -p -e "SELECT * FROM labdb.employees;"
```

Screenshot Evidence:

```
[ec2-user@ip-172-31-21-168 ~]$ mysql -h mariadb-rds.cy1guuygysx7.us-east-1.rds.amazonaws.com -P 3306 -u admin -p khadeer < khadeer_backup.sql
Enter password:
[ec2-user@ip-172-31-21-168 ~]$ mysql -h mariadb-rds.cy1guuygysx7.us-east-1.rds.amazonaws.com -P 3306 -u admin -p -e "SELECT COUNT(*) FROM khadeer.employees;"
Enter password:
+-----+
| COUNT(*) |
+-----+
|         5 |
+-----+
[ec2-user@ip-172-31-21-168 ~]$ mysql -h mariadb-rds.cy1guuygysx7.us-east-1.rds.amazonaws.com -P 3306 -u admin -p -e "SELECT * FROM khadeer.employees;"
Enter password:
+----+-----+-----+-----+
| id | name   | department | salary |
+----+-----+-----+-----+
| 1  | Ravi Kumar | Engineering | 65000.00 |
| 2  | Ananya Rao  | Finance    | 58000.00 |
| 3  | Suresh Reddy | Engineering | 72000.00 |
| 4  | Divya Sharma | HR         | 49000.00 |
| 5  | Kiran Patil | Sales      | 54000.00 |
+----+-----+-----+-----+
[ec2-user@ip-172-31-21-168 ~]$
```

Validation Steps

- SHOW DATABASES; on the RDS endpoint lists labdb.
- SELECT COUNT(*) FROM khadeer.employees; on RDS returns the same row count as the EC2 source.
- Spot-checking a few rows with SELECT * FROM khadeer.employees LIMIT 5; matches the source data.

Explanation

Piping the dump file directly at the RDS endpoint with the standard mysql client replays the same DDL and DML statements against the remote managed engine, requiring no special migration tooling for a small, static dataset. For larger datasets or migrations that must minimize downtime, AWS Database Migration Service (DMS) with change data capture is used instead.

Real-Time Use Case

This lightweight dump-and-restore approach is a common strategy for development and test-environment cutovers; production migrations that cannot tolerate extended downtime typically use AWS DMS to replicate ongoing changes while the cutover is staged.

Common Errors and Troubleshooting

Error	Cause	Resolution
ERROR 1044: Access denied for user 'admin'	Master username or password is incorrect, or the user lacks privileges on the target database	Confirm the RDS master credentials and re-run the restore command
Row counts differ between EC2 and RDS	Dump file restore was interrupted partway through	Re-run the restore, or re-create the dump file and restore again after confirming no errors were printed

Task 6: Install a MySQL Database on EC2

Outcome

MySQL server is installed and running on the EC2 instance as a second, independent source engine, with a populated database ready for the MySQL-focused RDS tasks.

Objective

To repeat the self-managed database setup pattern from Task 1 using the MySQL engine, providing the source system for the MySQL RDS tasks that follow.

Prerequisites

- A running EC2 instance with SSH or Session Manager access.
- Security group allowing inbound SSH (22) and, if needed, MySQL (3306).

Step-by-Step Implementation

Step 1: Install and Start MySQL Server

```
sudo yum install mysql-server -y
sudo systemctl start mysqld
sudo systemctl enable mysqld
sudo systemctl status mysqld
```

Screenshot Evidence:

```
[ec2-user@ip-172-31-21-168 ~]$ sudo dnf install -y https://dev.mysql.com/get/mysql84-community-release-e19-1.noarch.rpm
Last metadata expiration check: 2:51:19 ago on Mon Aug 3 12:51:03 2026.
mysql84-community-release-e19-1.noarch.rpm
Dependencies resolved.
63 kB/s | 13 kB | 00:00
=====
Package                                Architecture      Version           Repository        Size
=====
Installing:
mysql84-community-release              noarch            e19-1             @commandline      13 k
=====
Transaction Summary
=====
Install 1 Package

Total size: 13 k
Installed size: 14 k
Downloading Packages:
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing                :
  Installing               : mysql84-community-release-e19-1.noarch 1/1
  Verifying                : mysql84-community-release-e19-1.noarch 1/1
Installed:
mysql84-community-release-e19-1.noarch

Complete!
[ec2-user@ip-172-31-21-168 ~]$ sudo yum install mysql-community-server -y
MySQL 8.4 LTS Community Server
MySQL Connectors Community
MySQL Tools 8.4 LTS Community
Dependencies resolved.
60 MB/s | 2.3 MB | 00:00
6.7 MB/s | 123 kB | 00:00
41 MB/s | 1.1 MB | 00:00
=====
Package                                Architecture      Version           Repository        Size
=====
```

```
[ec2-user@ip-172-31-21-168 ~]$ sudo dnf install -y mysql-community-server
Amazon Linux 2023 repository
Amazon Linux 2023 kernel Livepatch repository
MySQL 8.4 LTS Community Server
MySQL Connectors Community
MySQL Tools 8.4 LTS Community
Dependencies resolved,
=====
Package                                Architecture      Version           Repository         Size
-----
Installing:
mysql-community-server                 x86_64            8.4.11-1.el9      mysql-8.4-lts-community 52 M
Installing dependencies:
mysql-community-client                 x86_64            8.4.11-1.el9      mysql-8.4-lts-community 3.4 M
mysql-community-client-plugins         x86_64            8.4.11-1.el9      mysql-8.4-lts-community 1.5 M
mysql-community-common                 x86_64            8.4.11-1.el9      mysql-8.4-lts-community 581 k
mysql-community-icu-data-files         x86_64            8.4.11-1.el9      mysql-8.4-lts-community 2.3 M
mysql-community-libs                   x86_64            8.4.11-1.el9      mysql-8.4-lts-community 1.5 M
=====
Transaction Summary
-----
Install 6 Packages

Total download size: 62 M
Installed size: 340 M
Downloading Packages:
(1/6): mysql-community-common-8.4.11-1.el9.x86_64.rpm 32 MB/s | 581 kB 00:00
(2/6): mysql-community-client-plugins-8.4.11-1.el9.x86_64.rpm 48 MB/s | 1.5 MB 00:00
(3/6): mysql-community-icu-data-files-8.4.11-1.el9.x86_64.rpm 68 MB/s | 2.3 MB 00:00
(4/6): mysql-community-libs-8.4.11-1.el9.x86_64.rpm 50 MB/s | 1.5 MB 00:00
(5/6): mysql-community-client-8.4.11-1.el9.x86_64.rpm 24 MB/s | 3.4 MB 00:00
(6/6): mysql-community-server-8.4.11-1.el9.x86_64.rpm 147 MB/s | 52 MB 00:00
-----
Total
146 MB/s | 62 MB 00:00
Running transaction check
Transaction check succeeded.
Running transaction test

sudo systemctl start mysqld
[ec2-user@ip-172-31-21-168 ~]$ systemctl status mysqld
● mysqld.service - MySQL Server
   Loaded: loaded (/usr/lib/systemd/system/mysqld.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-08-03 15:49:37 UTC; 14s ago
     Docs: man:mysqld(8)
           http://dev.mysql.com/doc/refman/en/using-systemd.html
   Process: 34429 ExecStartPre=/usr/bin/mysqld_pre_systemd (code=exited, status=0/SUCCESS)
  Main PID: 34456 (mysqld)
    Status: "Server is operational"
     Tasks: 35 (limit: 1059)
   Memory: 441.2M
      CPU: 1.696s
   CGroup: /system.slice/mysqld.service
           └─34456 /usr/sbin/mysqld

Aug 03 15:49:35 ip-172-31-21-168.ec2.internal systemd[1]: Starting mysqld.service - MySQL Server...
Aug 03 15:49:37 ip-172-31-21-168.ec2.internal systemd[1]: Started mysqld.service - MySQL Server.
[ec2-user@ip-172-31-21-168 ~]$
```

Explanation

MySQL Community Server is installed and administered the same way as any self-managed database, requiring manual patching, backup scheduling, and high-availability configuration. This mirrors Task 1 but establishes MySQL specifically as the source engine, since the target RDS engine in Task 7 onward is MySQL rather than MariaDB.

Real-Time Use Case

Running both engines side by side on the same lab instance mirrors real migration projects where an organization evaluates or migrates multiple database engines during a broader platform consolidation.

Common Errors and Troubleshooting

Error	Cause	Resolution
ERROR 1045: Access denied using password: YES	Temporary password from mysqld.log was mistyped or already expired	Re-check the log for the exact temporary password, or reset it with ALTER USER after connecting as root via socket auth

Task 7: Launch a MySQL RDS Instance

Outcome

A managed Amazon RDS instance running the MySQL engine is created and reaches the Available state.

Objective

To provision a managed MySQL target so the dataset from Task 6 can later be migrated, and so Multi-AZ, snapshot, and read-replica features can be exercised against a MySQL RDS instance.

Prerequisites

- IAM permissions to create RDS instances and associated networking resources.
- A DB subnet group spanning at least two Availability Zones.

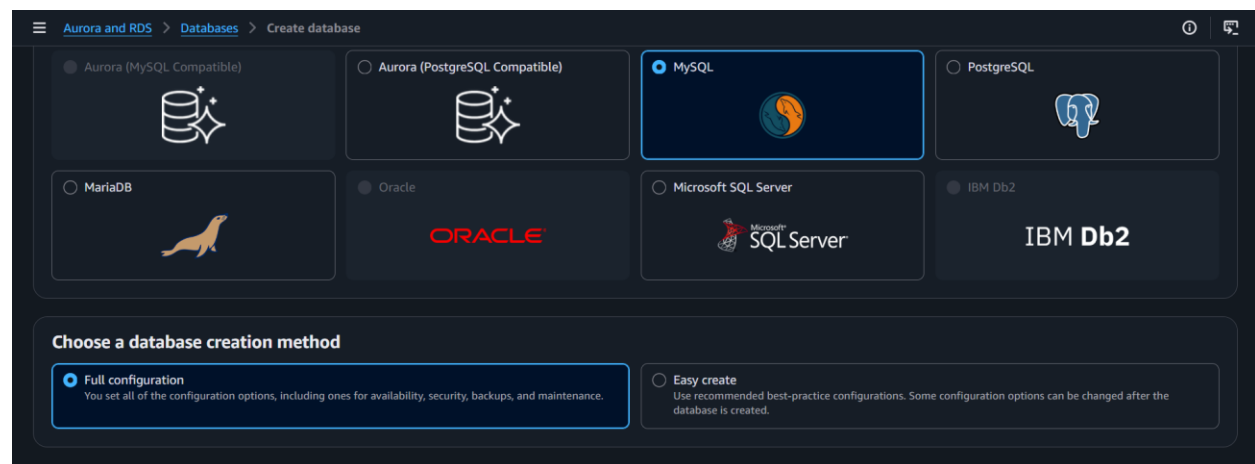
Step-by-Step Implementation

Step 1: Open the RDS Create Database Wizard

RDS Console -> Databases -> Create database -> Standard create, then configure:

Parameter	Value
Engine type	MySQL
Engine version	Latest supported minor version
Templates	Dev/Test (or Free tier if eligible)
DB instance identifier	mysql-rds
Master username	admin
DB instance class	db.t3.micro
Storage	20 GiB, gp2
Multi-AZ deployment	No (configured separately in Task 8)
VPC security group	Allow inbound 3306 from the EC2 instance's security group

Screenshot Evidence:



Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in the current AWS Region.

mysql-rds

The DB instance identifier is case-insensitive, but is stored as all lowercase (as in "mydbinstance"). Constraints: 1 to 63 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

Credentials Settings

Master username [Info](#)
Type a login ID for the master user of your DB instance.

admin

1 to 16 alphanumeric characters. The first character must be a letter.

Credentials management
You can use AWS Secrets Manager or manage your master user credentials.

☐ **Managed in AWS Secrets Manager - most secure**
RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.

☒ **Self managed**
Create your own password or have RDS create a password that you manage.

☐ **Auto generate password**
Amazon RDS can generate a password for you, or you can specify your own password.

Master password [Info](#)

DB subnet group [Info](#)
Choose the DB subnet group. The DB subnet group defines which subnets and IP ranges the DB instance can use in the VPC that you selected.

default-vpc-0a701668a4ff9329c
6 Subnets, 6 Availability Zones

Public access [Info](#)

☐ **Yes**
RDS assigns a public IP address to the database. Amazon EC2 instances and other resources outside of the VPC can connect to your database. Resources inside the VPC can also connect to the database. Choose one or more VPC security groups that specify which resources can connect to the database.

☒ **No**
RDS doesn't assign a public IP address to the database. Only Amazon EC2 instances and other resources inside the VPC can connect to your database. Choose one or more VPC security groups that specify which resources can connect to the database.

VPC security group (firewall) [Info](#)
Choose one or more VPC security groups to allow access to your database. Make sure that the security group rules allow the appropriate incoming traffic.

☒ **Choose existing**
Choose existing VPC security groups

☐ **Create new**
Create new VPC security group

Existing VPC security groups
Choose one or more options

default X mysg X

Availability Zone [Info](#)
us-east-1a

Step 2: Wait for the Instance to Become Available

Monitor the RDS console until the status changes to Available and note the endpoint hostname.

Screenshot Evidence:

Aurora and RDS > Databases > mysql-rds

Aurora and RDS < Dashboard Databases CloudWatch Database Insights Snapshots Exports in Amazon S3 Automated backups Reserved instances Proxies Subnet groups Parameter groups Option groups Custom engine versions Zero-ETL integrations Events Event subscriptions

mysql-rds Refresh Modify Actions

Summary

DB identifier mysql-rds	Status Available	Role Instance	Engine MySQL Community	Recommendations
CPU 7.24%	Class db.t4g.micro	Current activity 0 Connections	Region & AZ us-east-1a	

< **Connectivity & security** Monitoring Logs & events Configuration Zero-ETL integrations Maintenance & backups Data migration >

Connect using [Info](#)

☒ **Code snippets**
Use when connecting through SDK, APIs, or third-party tools including agents.

☐ **CloudShell**
Use for a quick access to AWS CLI that launches directly from the AWS Management Console.

☐ **Endpoints**
Use when connecting through any IDE interface.

Internet access gateway
☐ Disabled

IAM Authentication
☐ Disabled

Programming language
MySQL (macOS)

Endpoint type
Instance endpoint

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Validation Steps

- The instance status shows Available in the RDS console.
- A connection test from the EC2 instance to the RDS endpoint on port 3306 succeeds.

Explanation

This instance is created without Multi-AZ enabled so that enabling it in Task 8 is an observable, deliberate configuration change rather than a default, making the effect of Multi-AZ on the instance's configuration and behavior easier to see.

Real-Time Use Case

Standing up a baseline single-AZ instance before layering on Multi-AZ, snapshot restore, and read replicas mirrors how production databases are typically hardened incrementally after initial deployment rather than all at once.

Common Errors and Troubleshooting

Error	Cause	Resolution
DB subnet group doesn't meet Availability Zone coverage requirement	Subnet group has subnets in only one Availability Zone	Add a subnet from a second Availability Zone to the DB subnet group before creating the instance

Task 8: Configure Multi-AZ

Outcome

The MySQL RDS instance from Task 7 is modified to a Multi-AZ deployment, with a synchronous standby replica provisioned in a second Availability Zone.

Objective

To provide automatic failover and zero data-loss durability for the RDS instance, protecting against the loss of a single Availability Zone or an unplanned primary-instance failure.

Prerequisites

- Task 7 completed (mysql-rds instance Available).
- DB subnet group spans at least two Availability Zones.

Step-by-Step Implementation

Step 1: Open the Modify Wizard

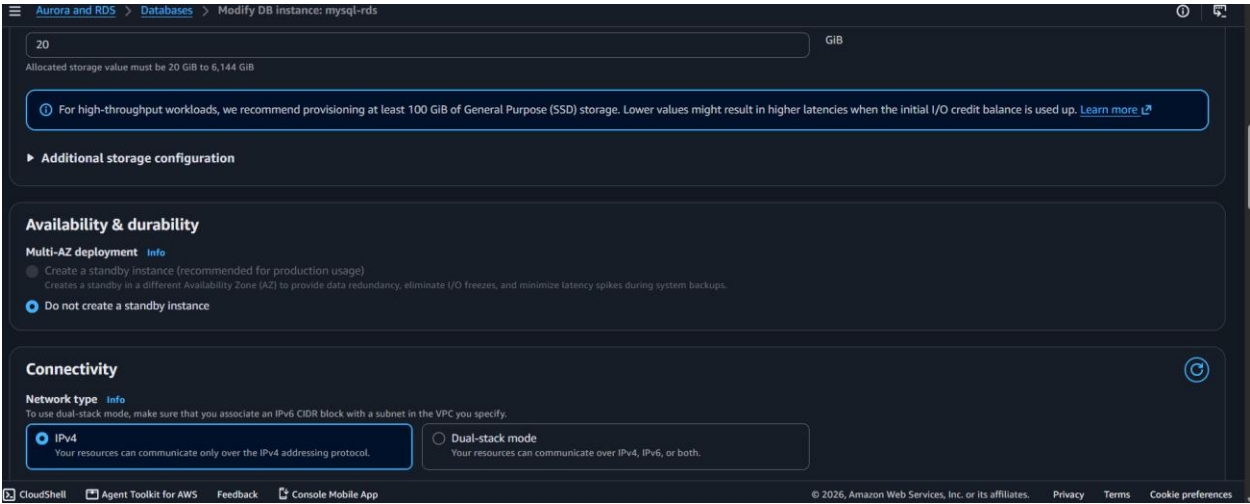
RDS Console -> Databases -> select mysql-rds -> Modify.

Step 2: Enable Multi-AZ Deployment

Parameter	Value
Multi-AZ deployment	Yes (creates a standby instance)
Apply immediately	Yes (for lab purposes)

Step 3: Multi-AZ Deployment is not available for free tier AWS account so we cannot Do this task

Screenshot Evidence:



Validation Steps

- The instance's Configuration tab shows Multi-AZ: Yes.
- A secondary Availability Zone is listed alongside the primary in the instance details.

Explanation

Multi-AZ maintains a synchronous physical standby replica of the entire instance in a different Availability Zone. Writes are committed to both the primary and standby before being acknowledged, giving a recovery point objective of zero. On primary failure, RDS automatically fails over to the standby and updates the DNS endpoint, typically within about one to two minutes. Multi-AZ exists purely for availability and durability; unlike a read replica, the standby is not directly queryable.

Real-Time Use Case

Production database tiers enable Multi-AZ so that an Availability Zone outage, instance hardware failure, or routine patching maintenance triggers an automatic failover instead of requiring manual intervention and extended downtime.

Common Errors and Troubleshooting

Error	Cause	Resolution
Modification stays in 'Applying' for an extended period	Standby provisioning and initial synchronization take time proportional to data volume	Wait for completion; avoid interrupting the modification while it is in progress
Brief connectivity interruption during failover testing	Expected DNS-based failover behavior when the primary is forced to fail over	Ensure application connection logic retries transient connection errors instead of treating them as fatal

Task 9: Take a Backup of the DB and Restore the DB

Outcome

A manual snapshot of the MySQL RDS instance is created, and a new RDS instance is successfully restored from that snapshot with matching data.

Objective

To validate that RDS's storage-level backup and restore mechanism produces a fully working, independent copy of the database, usable for recovery or as a safe testing copy.

Prerequisites

- Task 7 (and optionally Task 8) completed, with data present in mysql-rds-lab.
- IAM permissions to create RDS snapshots and restore new instances from them.

Step-by-Step Implementation

Step 1: Take a Manual Snapshot

RDS Console -> Databases -> select mariadb-rds -> Actions -> Take snapshot.

Parameter	Value
Snapshot identifier	mariadb-rds -manual-snapshot

Screenshot Evidence:

The screenshot shows the 'Take DB Snapshot' dialog in the AWS RDS console. The 'DB instance' dropdown is set to 'mariadb-rds'. The 'Snapshot name' text input field contains 'mariadb-rds -manual-snapshot'. The dialog includes a 'Cancel' button and a yellow 'Take snapshot' button.

Step 2: Wait for the Snapshot to Complete

RDS Console -> Snapshots. Wait until the snapshot status changes from Creating to Available.

Screenshot Evidence:

The screenshot shows the 'Manual snapshots (1)' list in the AWS RDS console. The table has columns for DB instance or cluster, Snapshot creation time, DB Instance created time, Status, Progress, and VPC. One snapshot is listed with a status of 'Available...' and 'Completed'.

DB instance or cluster	Snapshot creation time	DB Instance created time	Status	Progress	VPC
mysql-rds	August 03, 2026, 21:44 (UTC+05:30)	August 03, 2026, 21:33 (UTC+05:30)	Available...	Completed	vpc-0a70166

Step 3: Restore a New Instance from the Snapshot

Select the snapshot -> Actions -> Restore snapshot, then configure:

Parameter	Value
New DB instance identifier	mysql-rds-restored
DB instance class	db.t3.micro
VPC / security group	Same VPC; security group allowing 3306 from EC2

Screenshot Evidence:

The screenshot shows the AWS Management Console interface for restoring a snapshot. The top navigation bar indicates the path: Aurora and RDS > Snapshots > Restore snapshot. Below this, there are diagrams illustrating the restoration process: a primary instance with SSD, a readable standby with SSD in AZ 3, a primary instance, a standby instance, and a primary instance. The 'Settings' section shows the 'DB snapshot ID' as 'mysql-rdb-backup-1' and the 'DB instance identifier' as 'mysql-rds-restored'. The 'Instance configuration' section shows the 'DB instance class' as 'db.t3.micro'. A green banner at the top of the instance details page states: 'Snapshot restored successfully. Restoring mysql-rdb-backup to mysql-rds-restored was successful.' Below this, the instance details for 'mysql-rds-restored' are shown, including its status as 'Available', CPU usage at 6.78%, and other configuration details. The 'Connectivity & security' tab is selected, showing options for 'Code snippets', 'CloudShell', and 'Endpoints', all of which are currently disabled.

Settings

DB snapshot ID
The identifier for the DB snapshot.
mysql-rdb-backup-1

DB instance identifier Info
Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in the current AWS Region.
mysql-rds-restored
The DB instance identifier is case-insensitive, but is stored as all lowercase (as in "mydbinstance"). Constraints: 1 to 63 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

Instance configuration
The DB instance configuration options below are limited to those supported by the engine that you selected above.
DB instance class Info

mysql-rds-restored Refresh Refresh Modify Actions

Summary

DB identifier mysql-rds-restored	Status Available	Role Instance	Engine MySQL Community	Recommendations
CPU 6.78%	Class db.t3.micro	Current activity 0 Connections	Region & AZ us-east-1a	

Connectivity & security **Monitoring** **Logs & events** **Configuration** **Zero-ETL integrations** **Maintenance & backups** **Data migration**

Connect using Info

☒ **Code snippets**
Use when connecting through SDK, APIs, or third-party tools including agents.

☐ **CloudShell**
Use for a quick access to AWS CLI that launches directly from the AWS Management Console.

☐ **Endpoints**
Use when connecting through any IDE interface.

Internet access gateway
☐ Disabled

IAM Authentication
☐ Disabled

Validation Steps

- The restored instance mysql-rds-restored reaches status Available with its own new endpoint.
- `SELECT COUNT(*) FROM mysqlabdb.employees;` on the restored instance returns the same count as the original.
- Row-level data on the restored instance matches the original instance at the time the snapshot was taken.

Explanation

RDS snapshots are storage-level, point-in-time backups of the entire DB instance (both automated daily snapshots and manual on-demand snapshots use the same underlying mechanism). Restoring a snapshot always creates a new DB instance with a new endpoint; RDS does not support restoring data back into the original instance in place. Any restore, whether to recover from an error or to stage a copy for testing, therefore results in a second, independent instance.

Real-Time Use Case

Manual snapshots are commonly taken immediately before a risky schema migration or application deployment as a safety net, and snapshot restores are used to spin up a read-only or throwaway copy of production data for reporting, debugging, or testing without touching the live instance.

Common Errors and Troubleshooting

Error	Cause	Resolution
Cannot connect to the newly restored instance	The restored instance's security group does not automatically inherit inbound rules	Attach or configure a security group on the restored instance that allows inbound 3306 from the client
Restore takes noticeably longer than expected	Restore time scales with the size of the snapshot and the storage type	Allow additional time for larger datasets; this is expected behavior, not an error

Task 10: Create a Read Replica

Outcome

A read replica of the primary MySQL RDS instance is created, asynchronously replicating from the primary and reachable on its own distinct read-only endpoint.

Objective

To offload read-only query traffic from the primary instance and demonstrate RDS's native asynchronous replication feature, distinct from the synchronous Multi-AZ standby configured in Task 8.

Prerequisites

- Task 7 completed, with automated backups enabled on the primary instance (required for read replicas).
- IAM permissions to create RDS read replicas.

Step-by-Step Implementation

Step 1: Confirm Automated Backups Are Enabled

RDS Console -> select mysql-rds -> Maintenance & backups -> confirm a backup retention period greater than zero is set; read replicas cannot be created without this.

Screenshot Evidence:

Backup

Automated backups
Enabled (1 Day)

Copy tags to snapshots
Enabled

Backup target
AWS Cloud (US East (N. Virginia))

Latest restore time
August 03, 2026, 23:16 (UTC+05:30)

Backup window
10:09-10:39 UTC (GMT)

Replicate to Region
-

Replicated automated backup
-

Snapshots (2)

Restore

Remove

Take snapshot

Q Filter by snapshots

< 1 > ⚙

☐	Snapshot name	Snapshot creation time	Status	Snapshot type
☐	mysql-rdb-backup	August 03, 2026, 21:44 (UTC+05:30)	Available	Manual
☐	rds:mysql-rds-2026-08-03-16-04	August 03, 2026, 21:34 (UTC+05:30)	Available	Automated

Step 2: Create the Read Replica

Select mysql-rds--> Actions -> Create read replica, then configure:

Parameter	Value
DB instance identifier	mysql-rds-replica
DB instance class	db.t3.micro
Multi-AZ	No (single-AZ replica for this lab)
Public access	Match the primary's setting

Step 3: Wait for the Replica to Become Available

Monitor the RDS console until the replica status changes to Available and note its distinct endpoint.

Screenshot Evidence:

Replica source

Source DB instance identifier

mysql-rds

Role: Instance

DB instance identifier

This is the unique key that identifies a DB instance. This parameter is stored as a lowercase string (for example, mydbinstance).

mysql-rds-replica

Instance configuration

The DB instance configuration options below are limited to those supported by the engine that you selected above.

DB instance class

Info

Hide filters

☐ Include previous generation classes

☐ Standard classes (includes m classes)

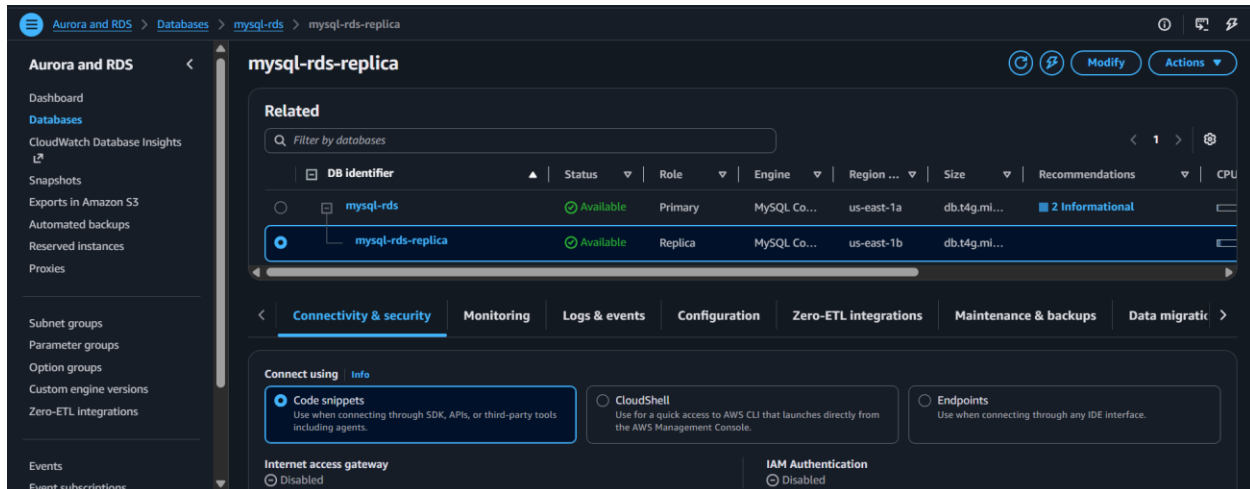
☐ Memory optimized classes (includes r and x classes)

☒ Burstable classes (includes t classes)

Instance type

db.t4g.micro

2 vCPUs 1 GiB RAM EBS Bandwidth: Up to 2,085 Mbps Network: Up to 5 Gbps



Validation Steps

- The replica reaches status Available and shows Replication source: mysql-rds in the console.
- The ReplicaLag CloudWatch metric is at or near zero under normal conditions.
- `SELECT COUNT(*) FROM khadeer.employees;` against the replica endpoint returns the same count as the primary.
- An attempted write (`INSERT`) against the replica endpoint fails, confirming it is read-only.

Explanation

Read replicas use MySQL's native asynchronous replication to stream changes from the primary's binary log to one or more replica instances, each with its own distinct, directly queryable endpoint. This differs from a Multi-AZ standby, which replicates synchronously purely for failover and cannot be queried directly. A read replica can also be promoted to a standalone, independently writable instance if needed.

Real-Time Use Case

Read replicas are used to scale read-heavy workloads such as reporting dashboards and analytics queries without adding load to the primary instance, and a cross-region read replica is a common lightweight disaster-recovery option that can be promoted to a full primary if the original region becomes unavailable.

Common Errors and Troubleshooting

Error	Cause	Resolution
Create read replica option is disabled/grayed out	Automated backups are disabled on the primary instance	Set the backup retention period to a value greater than zero on the primary, then retry
ReplicaLag climbs steadily instead of staying near zero	Write volume on the primary exceeds what the replica's instance class can apply	Scale the replica to a larger instance class or reduce write volume on the primary